"بسم الله الرحمن الرحيم"

Faculty of Engineering & Technology

Electrical & Computer Engineering Department

ENCS5341, Machine Learning and Data Science

Assignment 3

Prepared by:

Mohammad Shamasneh	**ID: 1220092**	**Sec: 1**
Khaled Hammad	**ID: 1220857**	**Sec: 1**

Instructor: Dr. Yazan Abu Farha

Date of submission: 1/13/2026

1. Introduction

In this project, we built and compared models for two classification tasks using the same dataset. For **Season classification from images** (Fall/Spring/Summer/Winter), we compared kNN and **Random Forest** using **RGB color histograms**, **Random Forest** using **ResNet18-extracted deep features**, and a **fine-tuned ResNet18** transfer learning model. For **TimeOfDay classification from text** (Morning/Afternoon/Evening), we used **TF-IDF** features and compared **kNN**, **Logistic Regression(softmax)**, and **Linear SVM** with hyperparameters selected on a validation set.

1.2. Task Description

1.2.1 predict the time of day

The selected task is to **predict the time of day** associated with a travel description.

- **Input:** Textual description of a travel destination
- **Output:** TimeOfDay label ∈ {Morning, Afternoon, Night}
- **Problem type:** Multi-class classification

1.2.2 predict the season

The selected task is to **predict the season** associated with Images.

- **Input:** Image (from ImageURL / local image path)
- **Output:** Season label ∈ {Summer, Fall, Spring, Winter}
- **Problem type:** Multi-class classification

1.3. Evaluation Metrics

The following evaluation metrics are used for all models:

- **Accuracy:** Overall proportion of correct predictions
- **Precision, Recall, and F1-score:**
  These metrics treat all classes equally and are appropriate for multi-class classification.
- **Confusion Matrix:**
  Used to analyze specific patterns of misclassification.

2. Data Preprocessing and Exploratory Data Analysis

2.1 Data Preprocessing

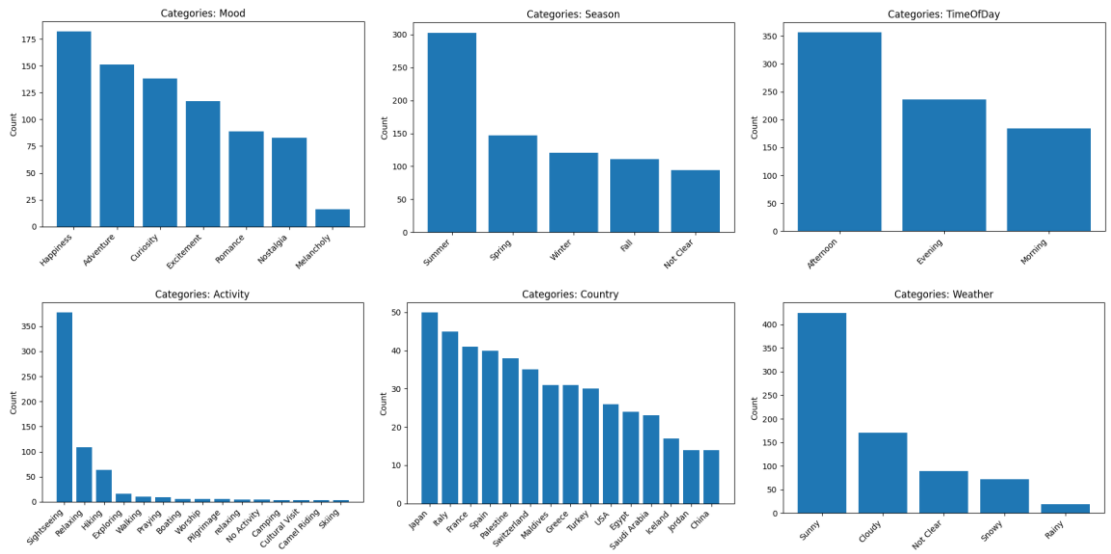
The dataset was preprocessed as follows:

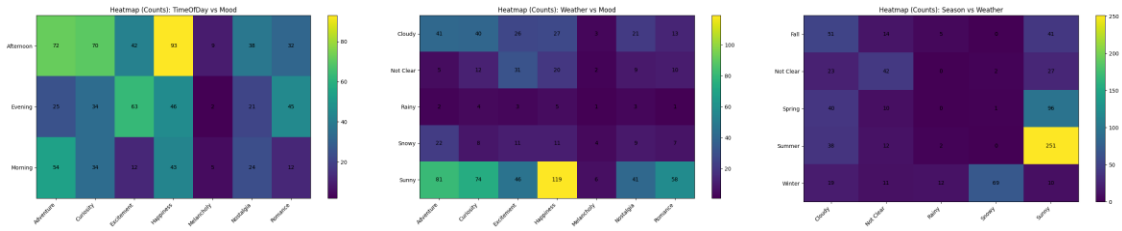
- We take the good row from a 121 CSV file, # of good row we find: 845
- Then we remove the Duplicated rows (68 row), # of row become 777

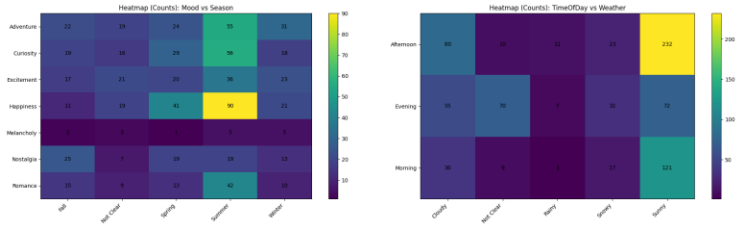
- We modify manually the data (missing value, incorrect data, standardized feature to three categories)
- For image task, checked the URL and removing the wrong row (invalid URL) then converting the ImageURL to local image path and download the images.
- For text descriptions task we converted description to lowercase, remove Punctuation, numbers, and extra whitespace.

After preprocessing, the cleaned dataset was used for all experiments.

2.2 Exploratory Data Analysis








```
========== INFO ==========
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 776 entries, 0 to 775
Data columns (total 8 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   ImageURL     776 non-null    object
 1   Description  776 non-null    object
 2   Country      776 non-null    object
 3   Weather      776 non-null    object
 4   TimeOfDay    776 non-null    object
 5   Season       776 non-null    object
 6   Activity     776 non-null    object
 7   Mood         776 non-null    object
dtypes: object(8)
memory usage: 48.6+ KB
None

========== MISSING VALUES ==========
             missing  missing_%
ImageURL           0        0.0
Description        0        0.0
Country            0        0.0
Weather            0        0.0
TimeOfDay          0        0.0
Season             0        0.0
Activity           0        0.0
Mood               0        0.0
```

This is the EDA figures of the clean data set, where the we use box plots to class distribution, and use heatmap for the correlations between feature.

NOTE: all feature extract from the image, so correlation influenced, and may not represent real-world relationships.

2.3. Trends or challenges

In the preprocessing we face many problems first problem is that they have a file without csv extension another problem that they have wrong representation to csv file another problem that they have wrong URL for image another problem that they have more than requirement label for example in TimeofDay the label must be in one of those (Morning, Afternoon, Evening) that is 3 labels but in data we have more than 20 label so we modified what can we modified in code then we modified with hand to reduce it to 3 label another problem was in weather we have a 37 sample that name (Clear) so we check the image one by one to label it to one of those(Sunny, Rainy, Cloudy, Snowy, Not Clear) and that is faced in other column.

3. Models and Experiments

Data Splitting

The dataset was split using stratified sampling into:

- **70% Training : 471**
- **15% Validation : 101**
- **15% Test : 101**

The training set was used to fit models, the validation set for hyperparameter tuning, and the test set for final evaluation.

A fixed random seed was used for making the split fixed.

3.1 Predict the Time of Day from Description

Feature extraction

Textual descriptions were converted into numerical features using **Term Frequency–Inverse Document Frequency (TF-IDF)**.

- TF-IDF assigns higher weights to informative words while down-weighting common terms.
- Unigrams and bigrams were used to capture both individual words and short phrases.
- A maximum number of features were selected to control dimensionality.

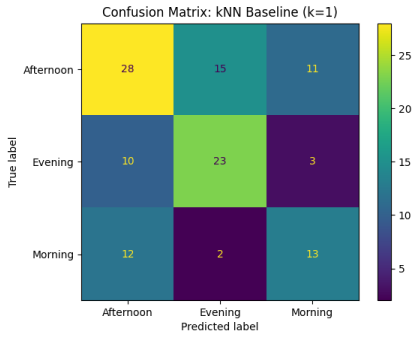
TF-IDF is well suited for short text descriptions and sparse feature representations.

3.1.1 Baseline Model: k-Nearest Neighbors

A k-Nearest Neighbors (kNN) classifier was used as the baseline model.

- Distance metric: cosine distance
- Tested values: k = 1 and k = 3

K=1



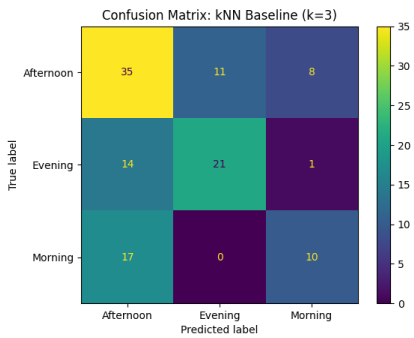

```
===== kNN Baseline (k=1) =====
Accuracy: 0.5470085470085471

Classification report:
              precision    recall  f1-score   support

   Afternoon       0.56      0.52      0.54        54
     Evening       0.57      0.64      0.61        36
     Morning       0.48      0.48      0.48        27

    accuracy                           0.55       117
   macro avg       0.54      0.55      0.54       117
weighted avg       0.55      0.55      0.55       117
```

K=3




```
===== kNN Baseline (k=3) =====
Accuracy: 0.5641025641025641

Classification report:
              precision    recall  f1-score   support

   Afternoon       0.53      0.65      0.58        54
     Evening       0.66      0.58      0.62        36
     Morning       0.53      0.37      0.43        27

    accuracy                           0.56       117
   macro avg       0.57      0.53      0.55       117
weighted avg       0.57      0.56      0.56       117
```

The results show that kNN provides a reasonable baseline but is limited in its ability to generalize compared to more advanced models.

3.1.2 Logistic Regression

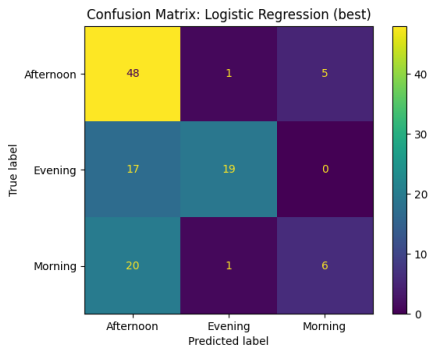
Logistic Regression was selected due to its effectiveness in text classification tasks and its interpretability.

- Feature representation: TF-IDF
- Hyperparameter tuned: regularization parameter **C** that is invers of lambda
- Tested values: 0.01, 0.1, 1, 10

C	Validation Accuracy
0.01	0.457
0.1	0.466
1	0.655
10	0.621

The model was trained on the training set and evaluated on the validation set to select the best value of c that is as we see in figure below it is 1.

So, we choose C=1.




```
Best LR C: 1

===== Logistic Regression (best) =====
Accuracy: 0.6239316239316239

Classification report:
              precision    recall  f1-score   support

   Afternoon       0.56      0.89      0.69        54
     Evening       0.90      0.53      0.67        36
     Morning       0.55      0.22      0.32        27

    accuracy                           0.62       117
   macro avg       0.67      0.55      0.56       117
weighted avg       0.66      0.62      0.60       117
```

3.1.3 Linear Support Vector Machine

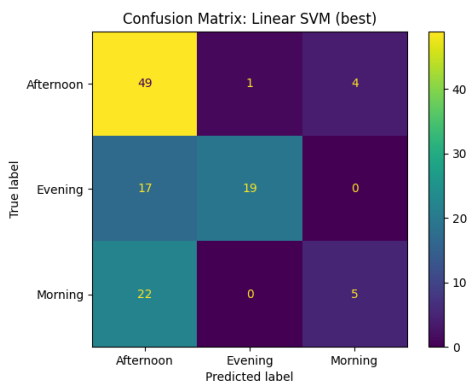
A Linear Support Vector Machine (SVM) was also evaluated due to its strong performance on sparse text features.

- Feature representation: TF-IDF
- Hyperparameter tuned: penalty parameter **C** that is put to penalty slack variable
- Tested values: 0.01, 0.1, 1, 10

So, we choose

C	Validation Accuracy
0.01	0.447
0.1	0.576
1	0.541
10	0.482

C = 0.1




```
Best SVM C: 0.1

===== Linear SVM (best) =====
Accuracy: 0.6239316239316239

Classification report:
              precision    recall  f1-score   support

   Afternoon       0.56      0.91      0.69        54
     Evening       0.95      0.53      0.68        36
     Morning       0.56      0.19      0.28        27

    accuracy                           0.62       117
   macro avg       0.69      0.54      0.55       117
weighted avg       0.68      0.62      0.59       117
```

3.1.4 Model Comparison

Overall, both Logistic Regression and Linear SVM outperform the kNN baseline. The Liner SVM and Logistic regression have same test accuracy but Logistic Regression is slightly better on F1.

3.1.5 Error Analysis

Error analysis was conducted on the best-performing model.

- Misclassifications often occur between **Morning** and **Afternoon**.
- There is a misclassification of data that is affect the accuracy.

3.2 predict the Season from Image

3.2.1 Baseline: KNN with RGB color histograms

A k-Nearest Neighbors (kNN) classifier was used as the baseline model. We tested two values of the main hyperparameter: k = 1 and k = 3.

Feature extraction: Each image is resized to **128×128**, then an **RGB color histogram** is computed with **HIST_BINS = 32** bins per channel. The histograms are **normalized** and concatenated, producing a **3 × HIST_BINS** feature vector (i.e., 96 features) for each image.

Before training **kNN**, we apply **StandardScaler** to the feature vectors because kNN is a **distance-based** method, scaling ensures that all features contribute more fairly to the distance computation.

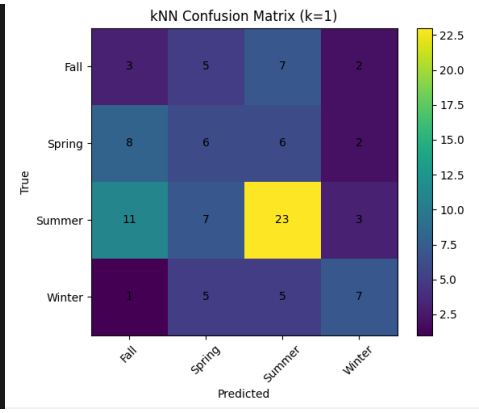
K = 1

```
===== kNN BASELINE (k=1, distance=euclidean) =====
Test Accuracy: 0.38613861386138615

Classification Report:
              precision    recall  f1-score   support

        Fall       0.13      0.18      0.15        17
      Spring       0.26      0.27      0.27        22
      Summer       0.56      0.52      0.54        44
      Winter       0.50      0.39      0.44        18

    accuracy                           0.39       101
   macro avg       0.36      0.34      0.35       101
weighted avg       0.41      0.39      0.40       101
```



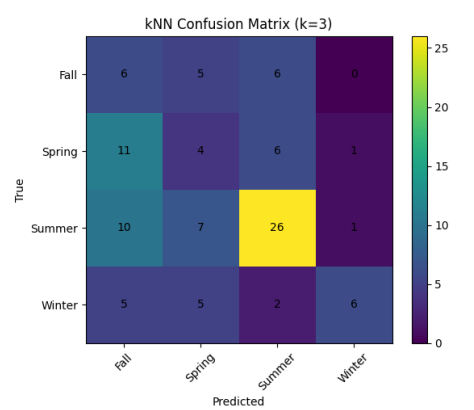
K = 3

```
===== kNN BASELINE (k=3, distance=euclidean) =====
Test Accuracy: 0.4158415841584158

Classification Report:
              precision    recall  f1-score   support

        Fall       0.19      0.35      0.24        17
      Spring       0.19      0.18      0.19        22
      Summer       0.65      0.59      0.62        44
      Winter       0.75      0.33      0.46        18

    accuracy                           0.42       101
   macro avg       0.44      0.36      0.38       101
weighted avg       0.49      0.42      0.43       101
```



The results show that kNN provides a simple baseline, but its performance is limited and it struggles to generalize, especially for classes such as Spring and Fall.

3.2.2 Random Forest with histogram features

We replaced the kNN baseline with a Random Forest (RF) classifier while keeping the same RGB color-histogram features. Unlike kNN, RF can learn non-linear decision boundaries and is generally more robust to noisy features.

Hyperparameter : **n_estimators** ∈ {200, 400, 800}, **max_depth** ∈ {None, 20, 40}, **min_samples_leaf** ∈ {1, 2, 4}. **Class weights** were set to **'balanced'** to mitigate class imbalance.

The model was trained on the training set and evaluated on the validation set to select the best value of hyperparameter that is as we see in figure below.

Performance is evaluated using **test accuracy**, **per-class Precision/Recall/F1**, and the **confusion matrix**.

```
RF cfg: n=200, depth=None, leaf=1 -> val_acc=0.5050
RF cfg: n=200, depth=None, leaf=2 -> val_acc=0.4950
RF cfg: n=200, depth=None, leaf=4 -> val_acc=0.4950
RF cfg: n=200, depth=20, leaf=1 -> val_acc=0.5050
RF cfg: n=200, depth=20, leaf=2 -> val_acc=0.4950
RF cfg: n=200, depth=20, leaf=4 -> val_acc=0.4950
RF cfg: n=200, depth=40, leaf=1 -> val_acc=0.5050
RF cfg: n=200, depth=40, leaf=2 -> val_acc=0.4950
RF cfg: n=200, depth=40, leaf=4 -> val_acc=0.4950
RF cfg: n=400, depth=None, leaf=1 -> val_acc=0.4950
RF cfg: n=400, depth=None, leaf=2 -> val_acc=0.5149
RF cfg: n=400, depth=None, leaf=4 -> val_acc=0.4752
RF cfg: n=400, depth=20, leaf=1 -> val_acc=0.4950
RF cfg: n=400, depth=20, leaf=2 -> val_acc=0.5149
RF cfg: n=400, depth=20, leaf=4 -> val_acc=0.4752
RF cfg: n=400, depth=40, leaf=1 -> val_acc=0.4950
RF cfg: n=400, depth=40, leaf=2 -> val_acc=0.5149
RF cfg: n=400, depth=40, leaf=4 -> val_acc=0.4752
RF cfg: n=800, depth=None, leaf=1 -> val_acc=0.4950
RF cfg: n=800, depth=None, leaf=2 -> val_acc=0.5149
RF cfg: n=800, depth=None, leaf=4 -> val_acc=0.4851
RF cfg: n=800, depth=20, leaf=1 -> val_acc=0.4950
RF cfg: n=800, depth=20, leaf=2 -> val_acc=0.5149
RF cfg: n=800, depth=20, leaf=4 -> val_acc=0.4851
RF cfg: n=800, depth=40, leaf=1 -> val_acc=0.4950
RF cfg: n=800, depth=40, leaf=2 -> val_acc=0.5149
RF cfg: n=800, depth=40, leaf=4 -> val_acc=0.4851

Best Histogram config: {'n_estimators': 400, 'max_depth': None, 'min_samples_leaf': 2} Best Val Acc: 0.5148514851485149
```

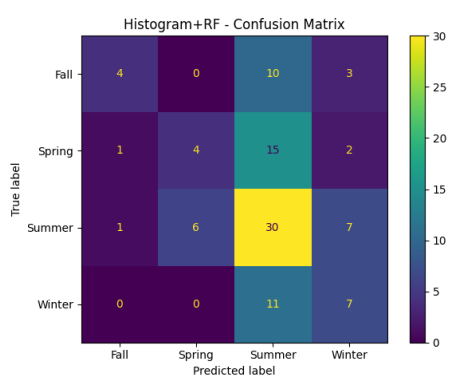
The best model when n_estimators = 400, max_depth = none, min_samples_leaf = 2

```
===== Histogram+RF | TEST RESULTS =====
Test Accuracy: 0.44554455445544555

Classification Report:
              precision    recall  f1-score   support

        Fall       0.67      0.24      0.35        17
      Spring       0.40      0.18      0.25        22
      Summer       0.45      0.68      0.55        44
      Winter       0.37      0.39      0.38        18

    accuracy                           0.45       101
   macro avg       0.47      0.37      0.38       101
weighted avg       0.46      0.45      0.42       101
```



The results show that Random Forest achieves better performance than kNN, but confusion between Spring and Fall remains a key limitation with histogram features.

3.2.3 Random Forest with feature from ResNet18 (CNN feature)

A pretrained **ResNet18** model was used as a fixed feature extractor by removing the final classification layer. Each image was resized to **224×224** and passed through the network to obtain a **512-dimensional** feature vector. A **Random Forest** classifier was then trained on these deep features instead of color-histogram features.

With the same tested hyperpar**a**meter.

```
RF cfg: n=200, depth=None, leaf=1 -> val_acc=0.5050
RF cfg: n=200, depth=None, leaf=2 -> val_acc=0.4851
RF cfg: n=200, depth=None, leaf=4 -> val_acc=0.5050
RF cfg: n=200, depth=20, leaf=1 -> val_acc=0.5050
RF cfg: n=200, depth=20, leaf=2 -> val_acc=0.4851
RF cfg: n=200, depth=20, leaf=4 -> val_acc=0.5050
RF cfg: n=200, depth=40, leaf=1 -> val_acc=0.5050
RF cfg: n=200, depth=40, leaf=2 -> val_acc=0.4851
RF cfg: n=200, depth=40, leaf=4 -> val_acc=0.5050
RF cfg: n=400, depth=None, leaf=1 -> val_acc=0.4950
RF cfg: n=400, depth=None, leaf=2 -> val_acc=0.5050
RF cfg: n=400, depth=None, leaf=4 -> val_acc=0.5050
RF cfg: n=400, depth=20, leaf=1 -> val_acc=0.4950
RF cfg: n=400, depth=20, leaf=2 -> val_acc=0.5050
RF cfg: n=400, depth=20, leaf=4 -> val_acc=0.5050
RF cfg: n=400, depth=40, leaf=1 -> val_acc=0.4950
RF cfg: n=400, depth=40, leaf=2 -> val_acc=0.5050
RF cfg: n=400, depth=40, leaf=4 -> val_acc=0.5050
RF cfg: n=800, depth=None, leaf=1 -> val_acc=0.4950
RF cfg: n=800, depth=None, leaf=2 -> val_acc=0.4950
RF cfg: n=800, depth=None, leaf=4 -> val_acc=0.4950
RF cfg: n=800, depth=20, leaf=1 -> val_acc=0.4950
RF cfg: n=800, depth=20, leaf=2 -> val_acc=0.4950
RF cfg: n=800, depth=20, leaf=4 -> val_acc=0.4950
RF cfg: n=800, depth=40, leaf=1 -> val_acc=0.4950
RF cfg: n=800, depth=40, leaf=2 -> val_acc=0.4950
RF cfg: n=800, depth=40, leaf=4 -> val_acc=0.4950

Best CNN config: {'n_estimators': 200, 'max_depth': None, 'min_samples_leaf': 1} Best Val Acc: 0.504950495049505
```

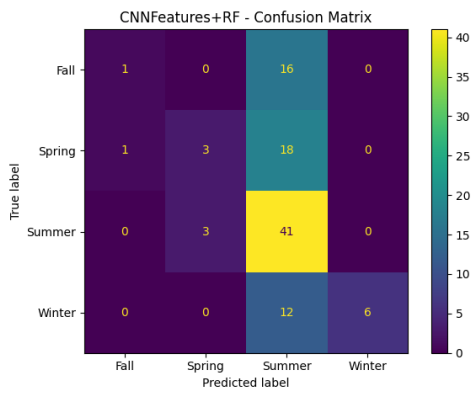
The best model when n_estimators = 200, max_depth = none, min_samples_leaf = 1

```
===== CNNFeatures+RF | TEST RESULTS =====
Test Accuracy: 0.504950495049505

Classification Report:
              precision    recall  f1-score   support

        Fall       0.50      0.06      0.11        17
      Spring       0.50      0.14      0.21        22
      Summer       0.47      0.93      0.63        44
      Winter       1.00      0.33      0.50        18

    accuracy                           0.50       101
   macro avg       0.62      0.37      0.36       101
weighted avg       0.58      0.50      0.43       101
```



The results show that Random Forest with ResNet18 features performs better than histogram-based RF, but Spring and Fall remain challenging and are often misclassified as Summer.

3.2.3 CNN Using ResNet18 model (end-to-end)

We used a **CNN transfer learning** approach based on a pretrained **ResNet18** for the 4-class season classification task (Fall/Spring/Summer/Winter). The model was adapted by replacing the final fully connected layer with a new layer that outputs **four classes**, and then the network was **fine-tuned** on our dataset. Images were resized to **224×224** and normalized using ImageNet mean and standard deviation. To improve generalization and

reduce overfitting, we applied **data augmentation** during training (random horizontal flips, small rotations, and slight color jitter). Training was performed using **cross-entropy loss** with the **Adam** optimizer with a small learning rate, for **8 epochs** as a **hyperparameter**, and the best checkpoint was selected based on **validation accuracy** before reporting final performance on the test set.

```
Epoch 1/8 | train_loss=1.3856 | train_acc=0.3312 | val_acc=0.4554
Epoch 2/8 | train_loss=0.7956 | train_acc=0.7473 | val_acc=0.4950
Epoch 3/8 | train_loss=0.5456 | train_acc=0.8471 | val_acc=0.4851
Epoch 4/8 | train_loss=0.3876 | train_acc=0.9066 | val_acc=0.5248
Epoch 5/8 | train_loss=0.2606 | train_acc=0.9427 | val_acc=0.5149
Epoch 6/8 | train_loss=0.1786 | train_acc=0.9660 | val_acc=0.5050
Epoch 7/8 | train_loss=0.1501 | train_acc=0.9809 | val_acc=0.5050
Epoch 8/8 | train_loss=0.1079 | train_acc=0.9766 | val_acc=0.4950

Best Validation Accuracy: 0.5247524752475248
```

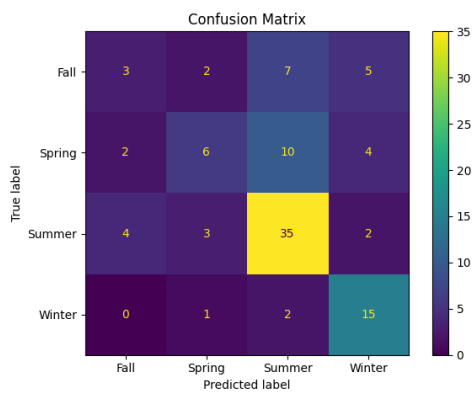
The best model with epoch = 4

```
===== TEST RESULTS =====
Test Accuracy: 0.5841584158415841

Classification Report:
              precision    recall  f1-score   support

        Fall       0.33      0.18      0.23        17
      Spring       0.50      0.27      0.35        22
      Summer       0.65      0.80      0.71        44
      Winter       0.58      0.83      0.68        18

    accuracy                           0.58       101
   macro avg       0.51      0.52      0.49       101
weighted avg       0.55      0.58      0.55       101
```



The results show that end-to-end fine-tuning of ResNet18 achieves the best overall performance, but the model still struggles with **Fall** and **Spring**, which are frequently confused with **Summer**, while **Summer** and **Winter** are classified more reliably.

3.2.4 Analysis

We analyzed errors of the best model (fine-tuned ResNet18) using the test-set classification report and confusion matrix. The model is strongest on **Summer** (**recall ≈ 0.80**) and **Winter** (**recall ≈ 0.83**), but it struggles with **Fall** (**recall ≈ 0.18**) and **Spring** (**recall ≈ 0.27**). Most errors are **Fall → Summer** and **Spring → Summer**, meaning the model often predicts Summer when seasonal cues are weak or ambiguous. This behavior is likely driven by **class imbalance** (Summer has the largest number of samples, creating a bias toward predicting it), the **visual similarity** between seasons in many images (e.g., indoor scenes, close-up objects, or landscapes without clear foliage changes), and **label noise** since the dataset was manually labeled by students and may contain some incorrect season assignments.

Conclusions and Discussion

In this project, we evaluated multiple models on two classification tasks using the same dataset: Season prediction from images (Fall/Spring/Summer/Winter) and TimeOfDay prediction from text descriptions (Morning/Afternoon/Evening).

Overall, the experiments show that **stronger feature representations lead to better generalization**: for the image task, moving from simple RGB histograms with kNN/RF to **deep features and transfer learning with ResNet18** produced the best performance, while for the text task, **linear models on TF-IDF features (Logistic Regression / Linear SVM)** outperformed the kNN baseline.

For season classification, the best overall approach was end-to-end fine-tuning of ResNet18, which improved results compared to histogram-based models; however, the model still struggled most with Fall and Spring, which were frequently confused with Summer, while Summer and Winter were classified more reliably.

The main limitations appear to come from **weak/ambiguous visual cues** (e.g., indoor scenes or images without clear seasonal patterns), **class imbalance** (Summer having more samples, encouraging a bias toward predicting it), and possible **label noise** because labels were manually assigned.

Finally, there are several clear directions to improve future performance: (1) collecting more balanced data across seasons or using stronger imbalance handling, (2) cleaning labels further or introducing quality checks to reduce annotation errors, and (3) using richer image information (e.g., higher-resolution training, stronger augmentations, or more advanced pretrained backbones). These improvements are especially important to reduce the persistent Fall/Spring confusion and make the model more robust across different locations and scene types.